

UNIVERSIDAD DE GUADALAJARA
Centro Universitario de Ciencias Exactas e Ingenierías
División de Tecnologías para la Integración Ciber-Humana



Algoritmos Metaheurísticos

Elizabeth Arroyo Moreno, Código: 221453749

José Luis Chavez Gomez, Código: 220419709

Karla Rebeca Hernández Elizarrarás, Código: 223991977

Ingeniería en Computación

Mtro. Carlos Alberto Villaseñor Padilla

Proyecto 1 El vendedor viajero de México

Septiembre del 2026

Introducción

Los algoritmos genéticos son técnicas de optimización inspiradas en la evolución biológica y la selección natural, útiles cuando el problema es demasiado grande como para revisar todas las soluciones posibles de forma exhaustiva. En esta actividad se pone a prueba lo visto en clase sobre el algoritmo genético ordinal, aplicándolo ahora a un caso real: encontrar la ruta más corta que recorra las 32 capitales de los estados de México, partiendo de datos geográficos y de tiempos de transporte reales entre ciudades.

A diferencia de un problema con ciudades aleatorias y distancia euclidiana, aquí cada cromosoma representa una permutación de capitales reales, y la función de costo utiliza el tiempo medio de transporte entre cada par de ciudades en lugar de una distancia geométrica. Esto acerca el problema a un escenario práctico, donde factores como el tipo de carretera o la disponibilidad de transporte afectan más al costo real de un recorrido que la simple distancia en línea recta.

El objetivo de esta actividad es aplicar los conceptos vistos en clase a este nuevo conjunto de datos, visualizando la mejor ruta encontrada sobre un mapa de México y analizando la evolución del costo total a lo largo de las generaciones.

Este tipo de problemas, conocidos como problemas de optimización combinatoria, aparecen con frecuencia en la vida real en contextos de logística, distribución y planeación de rutas, donde encontrar la solución exacta mediante fuerza bruta resulta computacionalmente imposible conforme crece el número de ciudades. Por ello, el uso de algoritmos genéticos como herramienta de búsqueda heurística permite obtener soluciones muy cercanas a la óptima en un tiempo razonable, sin necesidad de explorar todas las combinaciones posibles.

Código

```
# Pl EL vendedor viajero de México
# Algoritmos Metaheurísticos, Mtro. Carlos Carlos Alberto Villaseñor Padilla.
# -----
# Código: -----
# Karla Rebeca Hernandez Elizarraras
# Código: 223991977
# Elizabeth Arroyo Moreno
# Código: 221453749

import numpy as np
import matplotlib.pyplot as plt
import random
import pandas as pd
import geopandas
from shapely.geometry import Point

#-----
#Leer datos
#-----

# Se convierte la coordenada de texto a numero flotante,
# validando que este dentro del rango permitido

def convertir_coordenada(texto, minimo, maximo):
    texto = str(texto).strip()
    negativo = texto.startswith('-')
    digitos = texto.lstrip('-').replace('.', '')
    for n in (1, 2, 3):
        valor = float(digitos[:n] + '.' + digitos[n:])
        if minimo <= valor <= maximo:
            return -valor if negativo else valor
    raise ValueError(f"Coordenada no valida: {texto}")

# Leer los datos de las capitales y la matriz de tiempo de transporte

def leer_datos():
    capitales = pd.read_csv("Capitales.csv", dtype=str)
    capitales["Latitud"] = capitales["Latitud"].apply(lambda s: convertir_coordenada(s, 14,
33))
    capitales["Longitud"] = capitales["Longitud"].apply(lambda s: convertir_coordenada(s,
86, 119))
    capitales.set_index("Capital", inplace=True)
    tiempo = pd.read_csv("tiempo_transporte2.csv")
    tiempo.set_index("Origen", inplace=True) # filas = origen. columnas = destino
    return capitales, tiempo

# -----
# Funcion Dibujo
# -----

def dibujar_mexico(ax):
```

```

mexico =
geopandas.read_file("https://gist.githubusercontent.com/walkerke/76cb8cc5f949432f9555/raw/3
63c297ce82a4dcb9bdf003d82aa4f64bc695cf1/mx.geojson")
mexico.plot(ax=ax)

# Dibujar capitales en el mapa de Mexico

def dibujar_capitales(ax, capitales):
    geometry = [Point(xy) for xy in zip(capitales['Longitud'], capitales['Latitud'])]
    geo_df = geopandas.GeoDataFrame(capitales, geometry = geometry)
    geo_df.plot(ax=ax,color='red', marker='o', label='ciudad')

# Dibujar recorrido de la ruta
def dibujar_recorrido(perm, ax):
    df_temp = capitales.reindex(perm)
    return ax.plot(df_temp.Longitud, df_temp.Latitud, '-k')

# Datos de entrada: capitales y matriz de tiempo de transporte

# Leer datos
capitales, tiempo = leer_datos()
NOMBRES= list(capitales.index) # leer nombres de las capitales
NUM_CITIES= len(NOMBRES)      # numero de ciudades

MATRIZ_TIEMPO = tiempo.loc[NOMBRES, NOMBRES].values
# -----
# Parametros del AG
# -----

POP_SIZE = 300                # tamaño de la población
GENERATIONS = 1000            # número de generaciones
TOURNAMENT_PERCENT = 0.05     # porcentaje de la población que compite en cada torneo
TOURNAMENT_SIZE = max(2, int(POP_SIZE * TOURNAMENT_PERCENT))

#-----
# Funcion objetivo - distancia de la ruta cerrada
#-----

def tiempo_total(ruta):
    total_tiempo = 0
    for i in range(NUM_CITIES):
        total_tiempo += MATRIZ_TIEMPO[ruta[i], ruta[(i + 1) % NUM_CITIES]]
    return total_tiempo

#-----
# Funciones para el Algoritmo Genetico Ordinal
#-----

# Crear cromosoma : permutacion aleatoria de las ciudades
def crear_cromosoma():

```

```

    return random.sample(range(NUM_CITIES), NUM_CITIES)

# Crear poblacion inicial: lista de cromosomas
def crear_poblacion():
    return [crear_cromosoma() for _ in range(POP_SIZE)]

# Procesos reproductivos
# Operadores de reproduccion (mutacion sobre la representacion ordinal)

def proceso_reversa(padre): # Operador de reversa
    # toma un segmento aleatorio del cromosoma y lo invierte
    hijo = padre.copy()
    if len(hijo) > 2:
        i, j = sorted(random.sample(range(len(hijo)), 2))
        hijo[i:j + 1] = reversed(hijo[i:j + 1])
    return hijo

def proceso_enroque(padre): # Operador de insercion
    # toma un gen de una posicion aleatoria y lo inserta en otra
    # posicion aleatoria del cromosoma
    hijo = padre.copy()
    if len(hijo) > 2:
        i, j = sorted(random.sample(range(len(hijo)), 2))
        gen = hijo.pop(i)
        hijo.insert(j, gen)
    return hijo

#-----
# Algoritmo Genetico Ordinal
#-----
def algoritmo_genetico():
    poblacion = crear_poblacion() #crear poblacion inicial
    mejores_tiempos = []          # historial de la funcion de costo
    mejor_cromosoma_global = None
    mejor_tiempo_global = np.inf

    for g in range(GENERATIONS):
        # Evaluar todos los cromosomas de la poblacion actual
        tiempos = [tiempo_total(cromosoma) for cromosoma in poblacion]

        # Guardar el mejor individuo de esta generacion
        mejor_idx = np.argmin(tiempos)
        mejor_tiempo = tiempos[mejor_idx]
        mejores_tiempos.append(mejor_tiempo)

        if mejor_tiempo < mejor_tiempo_global:
            mejor_tiempo_global = mejor_tiempo
            mejor_cromosoma_global = poblacion[mejor_idx]

        # Generacion de hijos: seleccion por torneo + mutacion
        hijos = []

```

```

        for _ in range(POP_SIZE):
            participantes_idx = random.sample(range(POP_SIZE), TOURNAMENT_SIZE)
            tiempos_participantes = [tiempos[i] for i in participantes_idx]
            ganador = poblacion[participantes_idx[np.argmin(tiempos_participantes)]]

            # Aplicar mutacion al ganador para crear un hijo, se elige aleatoriamente entre
los dos operadores de mutacion
            if random.random() < 0.5:
                hijo = proceso_reversa(ganador)
            else:
                hijo = proceso_enroque(ganador)
            hijos.append(hijo)
        poblacion = hijos

        if g % 50 == 0:
            print(f"Generacion {g}: Mejor tiempo = {mejor_tiempo:.2f}")

        return mejor_cromosoma_global, mejores_tiempos, mejor_tiempo_global

# -----

if __name__ == "__main__":
    mejor_idx_ruta, historial, mejor_tiempo_global = algoritmo_genetico()
    print(f"\nMejor tiempo encontrado (record historico): {mejor_tiempo_global:.2f}")

    #Convertir indices de la mejor ruta a nombres de capitales
    mejor_ruta = [NOMBRES[i] for i in mejor_idx_ruta]
    print("Ruta:", " -> ".join(mejor_ruta + [mejor_ruta[0]]))

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))

    # Grafica 1: mejor ruta encontrada en Mexico
    # Dibujar mapa de Mx: capitales y recorrido (la 1era ciudas al final para cerrar el
ciclo)
    dibujar_mexico(axes[0])
    dibujar_capitales(axes[0], capitales)
    dibujar_recorrido(mejor_ruta + [mejor_ruta[0]], axes[0])
    axes[0].set_title(f"Mejor ruta encontrada\nTiempo = {mejor_tiempo_global:.2f}")

# Grafica 2: evolucion de la funcion de costo (historial por generacion)
    axes[1].plot(historial, color='crimson', label='Mejor tiempo por generacion')
    axes[1].axhline(mejor_tiempo_global, color='navy', linestyle='--',
                    label=f'Minimo global: {mejor_tiempo_global:.2f}')
    axes[1].set_title("Historial de la funcion de costo")
    axes[1].set_xlabel("Generacion")
    axes[1].set_ylabel("Tiempo total (costo)")
    axes[1].legend()

    plt.tight_layout()
    plt.savefig("resultados.png", dpi=150)
    plt.show()

```

Resultados

